

Bản trình chiếu: Cách giải tốt các bài toán thống kê động

Yu Jiangwei

Trường Trung học số 1 Trung Sơn, Quảng Đông

2007

Nguồn gốc: Solving dynamic statistics problems well

IOI China candidate team papers / 2007 / day 1 / Yu Jiangwei.ppt

1 Dẫn nhập

Trong thi tin học, các bài toán thống kê xuất hiện rất thường xuyên. Một ví dụ cơ bản là trên một dãy độ dài N , thực hiện M thao tác hỏi và cập nhật: hỏi giá trị lớn nhất/nhỏ nhất trên đoạn $[i, j]$, hoặc cộng đồng thời một giá trị c vào mọi phần tử của đoạn. Với N có thể tới 500 000 và số thao tác tới 20 000, cây đoạn cho lời giải tự nhiên với thời gian $\mathcal{O}(M \log N)$ và bộ nhớ $\mathcal{O}(N)$.

Cây đoạn thành công vì nó có cấu trúc tổ chức hiệu quả và hỗ trợ tốt thao tác trên đoạn. Nhưng tiền đề quan trọng là thứ tự giữa các phần tử của dãy không thay đổi. Khi đối tượng thống kê có thể bị đảo, chèn, xóa hoặc sắp xếp lại ở quy mô lớn, cây đoạn trở nên khó dùng; ta cần cấu trúc dữ liệu linh hoạt hơn.

2 Ví dụ 1: duy trì dãy, NOI 2005

Bài *Maintain Sequence* yêu cầu duy trì một dãy và hỗ trợ sáu thao tác:

- INSERT a : chèn một dãy mới sau phần tử thứ a ;
- DELETE a b : xóa từ phần tử thứ a đến thứ b ;
- MAKE-SAME a b c : gán mọi phần tử trong đoạn $[a, b]$ thành c ;
- REVERSE a b : đảo đoạn $[a, b]$;
- GET-SUM a b : hỏi tổng đoạn $[a, b]$;
- MAX-SUM: hỏi tổng lớn nhất của một đoạn con không rỗng.

Bài toán cần mô phỏng quá trình thay đổi của dãy, đồng thời luôn duy trì thông tin tổng và đoạn con tốt nhất. Cách mô phỏng bằng mảng hoặc danh sách liên kết chỉ lấy được một phần điểm. Lời giải chuẩn có thể dùng danh sách khối; slide trình bày thêm hướng dùng cấu trúc cây.

3 Biểu diễn dãy bằng cây tìm kiếm

Ý tưởng là dùng một cây tìm kiếm nhị phân để biểu diễn dãy: mỗi nút ghi một số, và kết quả duyệt trung thứ tự chính là dãy hiện tại. Khóa không phải là giá trị phần tử mà là vị trí ngầm của nó trong dãy.

Vấn đề then chốt là hầu hết thao tác đều tác động lên một *đoạn liên tục*. Trong cây, các nút của một đoạn liên tục có thể nằm rải rác. Ta cần một cách đưa các nút ấy gom lại thành một cây con để xử lý cùng lúc. Slide chọn splay tree vì thao tác truy cập có thể đưa nút vừa tìm lên gốc.

4 Nhắc lại splay tree

Splay tree là một BST tự điều chỉnh. Sau mỗi lần truy cập một nút, ta dùng các phép quay để đưa nút đó lên gốc. Ba nhóm quay được nhắc tới là:

- quay đơn Zig/Zag;
- quay đôi cùng hướng Zig-Zig/Zag-Zag;
- quay đôi khác hướng Zig-Zag/Zag-Zig.

Quá trình đưa nút lên gốc gọi là *splay*. Chi phí khấu hao của nó là $\mathcal{O}(\log N)$, và nhờ splay ta thực hiện được các thao tác cơ bản của BST. Với bài duy trì dãy, mỗi nút lưu thêm kích thước cây con *Size* để tìm phần tử thứ k . Thủ tục `SPLAYKTH` nhận một cây con và chỉ số k , rồi đưa nút thứ k trong thứ tự trung lên gốc cây con. Slide có đưa mã giả chi tiết cho các trường hợp trái/phải và các phép quay đôi; ở đây chỉ giữ lại ý tưởng vì phần mã khá dài và phụ thuộc cài đặt.

5 Chèn và tách đoạn

Sau khi thực hiện

$$Root \leftarrow \text{SPLAYKTH}(Root, K),$$

nút thứ K nằm ở gốc; $K - 1$ nút trước nó nằm trong cây con trái, còn $N - K$ nút sau nó nằm trong cây con phải. Vì vậy muốn chèn một đoạn sau vị trí K , ta đưa nút thứ K lên gốc rồi gắn cây biểu diễn đoạn mới vào giữa gốc và cây con phải.

Để tách đoạn $[a, b]$, slide dùng hai lần splay. Trước hết đưa nút sát phải của đoạn lên gốc, sau đó trong cây con trái của gốc, đưa nút sát trái lên gốc của cây con ấy. Khi đó toàn bộ đoạn $[a, b]$ bị cô lập trong một cây con cố định, nên các thao tác xóa, gán, đảo hoặc hồi tổng đều trở thành thao tác trên một cây con.

Để tránh nhiều trường hợp biên như $a = 1$ hoặc $b = N$, ta thêm hai nút rỗng ở đầu và cuối dãy; mọi vị trí nhập vào được dịch thêm 1.

6 Thông tin duy trì trên nút

Với DELETE, sau khi tách đoạn, ta có thể xóa trực tiếp cây con tương ứng. Với GET-SUM, mỗi nút chỉ cần duy trì tổng *Sum* của cây con.

Hai thao tác REVERSE và MAKE-SAME dùng cờ lười. Khi đảo, ta đổi hai con trái/phải và hoán đổi thông tin tiền tố/hậu tố; khi gán cùng giá trị, ta cập nhật toàn bộ thông tin của cây con theo kích thước và giá trị mới. Cờ được đẩy xuống trước khi truy cập con.

Với MAX-SUM, mỗi nút cần duy trì ba đại lượng chuẩn:

- *AllMax*: tổng đoạn con lớn nhất trong cây con;
- *LMax*: tổng lớn nhất của một đoạn bắt đầu từ phần tử đầu của cây con;
- *RMax*: tổng lớn nhất của một đoạn kết thúc tại phần tử cuối của cây con.

Các giá trị này được cập nhật bằng quy hoạch động khi ghép cây con trái, giá trị tại nút và cây con phải. Đáp án MAX-SUM là $AllMax$ ở gốc.

Khi nút có nhiều thông tin phụ và nhiều cờ lười, $SPLAYKTH$ phải được sửa để đẩy cờ ở các nút liên quan trước khi so sánh kích thước và quay. Slide có phiên bản mã giả thứ hai đánh dấu các dòng mới thêm; nội dung chính là luôn “push down” trước khi đi xuống hoặc quay.

7 Ví dụ 2: trình soạn thảo văn bản, NOI 2003

Bài *Text Editor* mô phỏng một chuỗi với các thao tác: INSERT, DELETE, GET, MOVE, PREV, NEXT. Tổng số ký tự được chèn có thể rất lớn, nhưng số thao tác chèn/xóa chỉ không quá 4000.

Nếu gom các thao tác di chuyển con trỏ lại và chỉ định vị khi cần, bài toán còn ba thao tác chính: chèn, xóa, lấy chuỗi. Ta có thể dùng splay như ví dụ 1, nhưng bài này có một cách đơn giản hơn nhờ chênh lệch lớn giữa số ký tự và số lần chèn/xóa.

Xem mỗi chuỗi được chèn như một khoảng liên tục trong một mảng lớn lưu ký tự theo thứ tự nhập. Sau đó các thao tác chỉ chia nhỏ các khoảng và thay đổi thứ tự các khoảng, chứ không cần di chuyển ký tự thật. Mỗi lần chèn/xóa tạo thêm nhiều nhất hai điểm chia mới; vì tổng số lần chèn/xóa không quá 4000, số khoảng không quá khoảng 8000. Đây chính là một dạng rời rạc hóa động theo đầu mút khoảng.

Khi số khoảng tăng quá lớn, ta có thể tái xây dựng: ghi các khoảng hiện tại theo thứ tự thành những khoảng lớn hơn. Một lần tái xây dựng tốn $\mathcal{O}(C)$, với C là độ dài chuỗi; nếu làm quá ít thì số khoảng lớn, nếu làm quá nhiều thì tái xây dựng thành nút cổ chai. Slide khuyến nghị dùng tư duy trung bình hình học để cân bằng hai phía; trong thực tế đặt ngưỡng số khoảng khoảng 1000 thường tốt.

8 Tổng kết

Muốn giải tốt bài toán thống kê động, cần chọn cấu trúc dữ liệu theo đúng kiểu thay đổi của đối tượng. Khi thứ tự cố định, cây đoạn và rời rạc hóa biên là đủ mạnh. Khi thứ tự thay đổi lớn, splay giúp gom đoạn liên tục thành cây con để cập nhật trong $\mathcal{O}(\log N)$ khâu hao. Khi số thao tác phá vỡ thứ tự ít hơn rất nhiều so với kích thước dữ liệu, rời rạc hóa bằng các khoảng động có thể đơn giản hơn hẳn. Thông điệp cuối của slide là phải phân tích điểm chung của thao tác, thử nhiều lời giải cho phiên bản đơn giản, và kiểm chứng bằng cài đặt thực tế.